QUBO Solver Functions and Optimisation

Quadratic Unconstrained Binary Optimisation (QUBO) was used to optimise collision-avoidance manoeuvres by formulating the decision problem as a binary optimisation task. The objective function balances collision risk reduction and manoeuvre cost, including fuel consumption and operational constraints. This formulation enables efficient evaluation of candidate manoeuvres using classical, quantum-inspired, and hybrid optimisation solvers [44].

Discrete Δv manoeuvre options were defined across three orbital axes: radial (R), tangential (T), and normal (N). The solver selected one manoeuvre per axis to minimise the quadratic objective function while satisfying feasibility constraints. Performance was evaluated using solution quality, runtime, and solver stability. This approach is widely used in combinatorial optimisation and mission planning problems due to its flexibility and computational efficiency [19], [44].

Code used:

Step 1: Imports and Configuration

The QUBO optimisation framework begins by importing required libraries for modelling, optimisation, and data processing. NumPy and Pandas are used for numerical operations and dataset handling. Dimod and D-Wave Ocean SDK libraries are imported to construct Binary Quadratic Models (BQMs). Matplotlib is used for visualisation, and optional hybrid solver libraries are included for quantum-inspired optimisation. Configuration parameters are defined to control optimisation behaviour such as manoeuvre limits, weighting coefficients, and solver settings.

```python
# Step 1: Imports & User Config

import pandas as pd
import numpy as np
import time
import math
import matplotlib.pyplot as plt
from functools import partial

import dimod
from dimod import BinaryQuadraticModel, SimulatedAnnealingSampler, ExactSolver

# LeapHybridSampler (Optional — requires D-Wave credentials)
try:
    from dwave.system import LeapHybridSampler
    HAS_LEAP = True
except Exception:
    HAS_LEAP = False

# User Config (FURSAN Dataset)

CDM_PATH = r"C:\Users\DELL\Desktop\PDE3823_Project_ShafaqMandha\outputs\high_risk.csv"

N_TEST = 20                              # Number of highest-risk conjunctions
DV_LEVELS_PER_AXIS = [0.0, 0.01, 0.03, 0.06]  # Discretized Δv magnitudes (m/s)
AXES = ["r", "t", "n"]                   # Radial, Tangential, Normal axes
MAX_DV_ALLOW = 0.3                       # Maximum allowable Δv (m/s)
PC_REDUCTION_SCALE = 5.0                 # Controls how Δv reduces collision probability

# QUBO Objective Weights
W_PC = 1000.0     # Collision risk penalty
W_DV = 50.0       # Fuel / maneuver cost
W_PAIR = 5.0      # Interaction penalty (optional)
```

Figure 1.1: QUBO Imports and Configuration

Step 2: Configuration Parameters

Utility functions were implemented to support QUBO formulation and collision modelling. A simplified exponential collision probability model was used to estimate post-manoeuvre collision risk. This simplified modelling approach enables fast evaluation of candidate manoeuvres while maintaining realistic behaviour for orbital collision prediction [18].

Helper functions were also developed to generate discrete manoeuvre variables and enforce one-hot constraints. These constraints ensure that only one Δv option is selected per axis. These utilities simplified QUBO construction and improved modularity.

```python
# Step 2: Utilities – Collision Model & Helpers

def collision_probability_after(pc0, dv_total, scale=PC_REDUCTION_SCALE):
    """
    Simple exponential model: Pc_after = pc0 * exp(-scale * dv_total)
    (tunable scale; validated in prototype)
    """
    return float(pc0) * math.exp(-scale * dv_total)

def make_variables_grid(axes=AXES, levels=DV_LEVELS_PER_AXIS):
    """
    Create list of variable names and mapping variable -> dv magnitude.
    One-hot encoding per axis, including zero impulse.
    """
    var_names = []
    var_to_dv = {}
    for ax in axes:
        for i, lvl in enumerate(levels):
            name = f"dv_{ax}_{i}"
            var_names.append(name)
            var_to_dv[name] = float(lvl)
    return var_names, var_to_dv

def add_onehot_penalty(bqm, axis_vars, gamma=50.0):
    """
    Add quadratic penalty gamma*(sum(x_i) - 1)^2 to BQM for variables in axis_vars.
    Ensures exactly one level per axis.
    """
    for v in axis_vars:
        bqm.add_variable(v, gamma * (1 - 2))  # linear term
    for i in range(len(axis_vars)):
        for j in range(i+1, len(axis_vars)):
            bqm.add_quadratic(axis_vars[i], axis_vars[j], 2 * gamma)
```

Figure 2.1: QUBO Configuration Parameters

Step 3: Load CDM and Pre-Processing

Utility functions are implemented to support QUBO formulation. A simplified exponential collision probability model is defined to estimate post-manoeuvre collision risk. Helper functions are created to generate discrete manoeuvre variables and enforce one-hot constraints.

```python
# Step 3: Load CDM & Pre-process

import os

# Safe CSV loading
if not os.path.exists(CDM_PATH):
    raise FileNotFoundError(f"CSV file not found at: {CDM_PATH}. Please check the path.")

cdm = pd.read_csv(CDM_PATH, dtype=str)  # Load as string for robustness

# Columns to convert to numeric
numeric_cols = [
    "relative_position_r", "relative_position_t", "relative_position_n",
    "relative_velocity_r", "relative_velocity_t", "relative_velocity_n",
    "risk", "max_risk_estimate", "miss_distance", "relative_speed"
]

for col in numeric_cols:
    if col in cdm.columns:
        cdm[col] = pd.to_numeric(cdm[col], errors="coerce")
    else:
        cdm[col] = np.nan  # Add column with NaN if missing

# Compute range and speed
cdm["range_m"] = np.sqrt(
    (cdm[["relative_position_r","relative_position_t","relative_position_n"]]**2).sum(axis=1)
)

cdm["speed_mps"] = np.sqrt(
    (cdm[["relative_velocity_r","relative_velocity_t","relative_velocity_n"]]**2).sum(axis=1)
)
```

```python
# Use `risk` as proxy for collision probability if collision_probability column is missing
if "collision_probability" in cdm.columns:
    cdm["Pc"] = cdm["collision_probability"].fillna(0.0)
else:
    cdm["Pc"] = cdm["risk"].fillna(0.0)

# Select top high-risk conjunctions
cdm_sorted = cdm.sort_values("Pc", ascending=False).head(N_TEST).reset_index(drop=True)
print(f"Selected {len(cdm_sorted)} conjunctions (top by Pc/risk).")

# Safe display
display_cols = ["Pc", "range_m", "speed_mps"]
if "event_id" in cdm_sorted.columns:
    display_cols.insert(0, "event_id")

cdm_sorted[display_cols]
```

Figure 3.1: Utility Functions and Collision Model

Step 4: QUBO Formulation for a Single Conjunction

A Binary Quadratic Model (BQM) was constructed for each conjunction event. Linear terms represented fuel cost and collision probability reduction, while quadratic terms captured interactions between manoeuvres across axes.

One-hot constraints were added to ensure valid manoeuvre selection. The resulting QUBO formulation enabled efficient optimization of collision-avoidance manoeuvres.

```python
# ==============================
# Step 4: Build QUBO for One Conjunction
# ==============================

# Generate variable names and DV mapping
var_names, var_to_dv = make_variables_grid(axes=AXES, levels=DV_LEVELS_PER_AXIS)

# Group variables per axis for one-hot constraints
axis_vars = {ax: [v for v in var_names if v.startswith(f"dv_{ax}_")] for ax in AXES}

def build_bqm_for_conjunction(pc0, w_pc=W_PC, w_dv=W_DV, w_pair=W_PAIR):
    """
    Build a BinaryQuadraticModel for a single conjunction with initial Pc = pc0.
    Decision selects exactly one level per axis, including 0-level.
    Objective: minimize Pc_after + w_dv * dv_total.
    """

    bqm = BinaryQuadraticModel({}, {}, 0.0, vartype='BINARY')

    # ------------------------------
    # Linear terms: propellant cost + approximate Pc reduction
    # ------------------------------
    for v, dv in var_to_dv.items():
        # Approximate linear Pc reduction: Pc reduction ~ w_pc * pc0 * scale * dv for small dv
        linear_pc_proxy = w_pc * pc0 * (1 - math.exp(-PC_REDUCTION_SCALE * dv))
        linear = w_dv * dv + linear_pc_proxy
        bqm.add_variable(v, linear)
```

```python
# ------------------------------
# Quadratic terms: capture combined Δv effect
# ------------------------------
s = PC_REDUCTION_SCALE
for i, vi in enumerate(var_names):
    for j in range(i+1, len(var_names)):
        vj = var_names[j]
        dv_i = var_to_dv[vi]
        dv_j = var_to_dv[vj]
        # Quadratic coefficient from second-order expansion
        coef = w_pc * pc0 * ( (s**2) / 2.0 ) * (dv_i * dv_j)
        bqm.add_quadratic(vi, vj, -coef)  # negative to reward combined DV in minimization

# ------------------------------
# One-hot penalties for each axis
# ------------------------------
gamma = max(10.0, w_dv * max(var_to_dv.values()) * 10.0)
for ax, vlist in axis_vars.items():
    for v in vlist:
        bqm.add_variable(v, bqm.linear.get(v, 0.0) - gamma)  # linear part of (sum - 1)^2
    for i in range(len(vlist)):
        for j in range(i+1, len(vlist)):
            bqm.add_quadratic(vlist[i], vlist[j], bqm.quadratic.get((vlist[i], vlist[j]),0.0) + 2 * gamma)

return bqm
```

Figure 4.1: Manoeuvre Variable Generation

Step 5: Solver Functions

Multiple QUBO solvers were implemented to evaluate optimisation performance across different algorithm classes. These solvers were grouped into:

Solver	Type	Purpose
Exact Solver	Classical (Exhaustive)	Used as a baseline to obtain the optimal solution for comparison with other solvers
Simulated Annealing (SA)	Classical Metaheuristic	Tested for efficient optimisation of large QUBO problems with good solution quality
Leap Hybrid	Quantum-Classical Hybrid	Evaluated to compare hybrid quantum performance with classical solvers
Greedy Solver	Classical Heuristic	Tested for fast decision-making and low computational cost
Random Solver	Baseline (Random Search)	Used as a lower-bound performance benchmark
Steepest Descent	Classical Local Search	Tested for iterative improvement and local optimisation capability
Simulated Quantum Annealing	Quantum-Inspired	Evaluated for quantum-inspired optimisation performance
Kerberos Hybrid	Hybrid (SA + Tabu + QPU)	Tested for advanced hybrid optimisation performance
QBSolv	Decomposition-Based	Evaluated for solving large-scale QUBO problems
Digital Annealer	Quantum-Inspired Hardware	Tested for high-speed combinatorial optimisation
Simulated Bifurcation	Physics-Inspired	Evaluated for alternative optimisation strategy
Genetic Algorithm	Evolutionary Algorithm	Tested for global search capability
Particle Swarm Optimisation (PSO)	Swarm Intelligence	Evaluated for population-based optimisation
Ant Colony Optimisation (ACO)	Swarm Intelligence	Tested for probabilistic path optimisation

Table 1: QUBO Solver Types Evaluated for Collision Risk Optimisation

These solvers enable comparison across classical, quantum-inspired, and hybrid optimisation approaches, which are increasingly used in aerospace optimisation problems [11], [13], [17].

```python
# 1. Exact Solver (Baseline)
def solve_exact(bqm):
    solver = ExactSolver()
    t0 = time.perf_counter()
    sampleset = solver.sample(bqm)
    t1 = time.perf_counter()
    return sampleset.first, t1 - t0

# 2. Simulated Annealing (SA)
def solve_simulated_annealing(bqm, num_reads=200):
    sampler = SimulatedAnnealingSampler()
    t0 = time.perf_counter()
    sampleset = sampler.sample(bqm, num_reads=num_reads)
    t1 = time.perf_counter()
    return sampleset.first, t1 - t0

# 3. Leap Hybrid (requires D-Wave)
def solve_leap_hybrid(bqm):
    if not HAS_LEAP:
        return None, None
    sampler = LeapHybridSampler()
    t0 = time.perf_counter()
    sampleset = sampler.sample(bqm)
    t1 = time.perf_counter()
    return sampleset.first, t1 - t0
```

```python
# 4. Greedy Solver
def solve_greedy(bqm):
    chosen = {}
    for ax, vlist in axis_vars.items():
        best_v, best_score = None, float('inf')
        for v in vlist:
            score = bqm.linear.get(v, 0.0)
            if score < best_score:
                best_score = score
                best_v = v
        chosen[best_v] = 1
    sample = {v: 0 for v in var_names}
    for v in chosen:
        sample[v] = 1
    energy = bqm.energy(sample)
    class FakeResult:
        def __init__(self, sample, energy):
            self.sample = sample
            self.energy = energy
    return FakeResult(sample, energy), 0.0

# 5. Random Solver
def solve_random(bqm):
    sample = {v: np.random.choice([0,1]) for v in var_names}
    energy = bqm.energy(sample)
    class FakeResult:
        def __init__(self, sample, energy):
            self.sample = sample
            self.energy = energy
    return FakeResult(sample, energy), 0.0
```

```python
# 6. Steepest Descent (simple local search)
def solve_steepest_descent(bqm, max_iter=100):
    sample = {v: 0 for v in var_names}
    best_energy = bqm.energy(sample)
    for _ in range(max_iter):
        improved = False
        for v in var_names:
            sample[v] ^= 1  # flip
            e = bqm.energy(sample)
            if e < best_energy:
                best_energy = e
                improved = True
            else:
                sample[v] ^= 1
        if not improved:
            break
    class FakeResult:
        def __init__(self, sample, energy):
            self.sample = sample
            self.energy = energy
    return FakeResult(sample, best_energy), 0.0

# 7-14. Stubs for advanced solvers (replace with library calls)
def solve_simulated_quantum_annealing(bqm): return None, None
def solve_kerberos_hybrid(bqm): return None, None
def solve_qbsolv(bqm): return None, None
def solve_digital_annealer(bqm): return None, None
def solve_simulated_bifurcation(bqm): return None, None
def solve_genetic_algorithm(bqm): return None, None
def solve_particle_swarm(bqm): return None, None
def solve_ant_colony(bqm): return None, None
```

```python
# ==============================
# Solver mapping for iteration
# ==============================
SOLVERS = {
    "Exact": solve_exact,
    "SA": solve_simulated_annealing,
    "LeapHybrid": solve_leap_hybrid,
    "Greedy": solve_greedy,
    "Random": solve_random,
    "SteepestDescent": solve_steepest_descent,
    "SimulatedQuantumAnnealing": solve_simulated_quantum_annealing,
    "KerberosHybrid": solve_kerberos_hybrid,
    "QBSolv": solve_qbsolv,
    "DigitalAnnealer": solve_digital_annealer,
    "SimulatedBifurcation": solve_simulated_bifurcation,
    "GeneticAlgorithm": solve_genetic_algorithm,
    "PSO": solve_particle_swarm,
    "ACO": solve_ant_colony,
}
```

Figure 5.1:Solver Execution

Step 6: Solver Evaluation on Selected Conjunctions

Each solver was evaluated on selected high-risk conjunction events. The Binary Quadratic Model was constructed for each event, and solvers generated candidate manoeuvres.

Performance metrics, including total Δv, post-manoeuvre collision probability, runtime, and feasibility, were computed. Results were stored and summarised for comparison across solvers.

```python
# Step 6: Evaluate solvers on selected conjunctions
# ==============================

results = []

for idx, row in cdm_sorted.iterrows():
    conj_id = row.get("conjunction_id", f"row{idx}")
    pc0 = float(row.get("Pc", 0.0))

    # Build BQM
    bqm = build_bqm_for_conjunction(pc0)

    solver_outcomes = {}

    # Run all solvers from Step 5
    for name, fn in SOLVERS.items():

        # Skip Leap if not available
        if name == "LeapHybrid" and not HAS_LEAP:
            solver_outcomes[name] = {"available": False}
            continue

        try:
            sol, runtime = fn(bqm)

            if sol is None:
                solver_outcomes[name] = {"available": False}
                continue

        except Exception as e:
            solver_outcomes[name] = {
                "available": False,
                "error": str(e)
            }
            continue
```

```python
        sample = sol.sample

        # Convert to dictionary
        if hasattr(sample, "items"):
            sample_dict = dict(sample)
        else:
            sample_dict = dict(sample)

        # Compute total DV
        dv_total = sum(
            var_to_dv[v] * int(sample_dict.get(v, 0))
            for v in var_names
        )

        # Compute Pc after maneuver
        pc_after = collision_probability_after(pc0, dv_total)

        # Feasibility
        feasible = dv_total <= MAX_DV_ALLOW

        # Energy
        energy = (
            float(sol.energy)
            if hasattr(sol, "energy")
            else bqm.energy(sample_dict)
        )

        solver_outcomes[name] = {
            "available": True,
            "sample": sample_dict,
            "dv_total": dv_total,
            "pc_after": pc_after,
            "runtime_s": runtime,
            "energy": energy,
            "feasible": feasible}
```

```python
    # Get exact solver energy (for optimality comparison)
    exact = solver_outcomes.get("Exact", {})
    exact_energy = exact.get("energy") if exact.get("available") else None

    # Store results
    results.append({
        "conjunction_id": conj_id,
        "Pc0": pc0,
        "exact_energy": exact_energy,
        "solver_outcomes": solver_outcomes
    })


# ==============================
# Create Summary Table
# ==============================

rows_summary = []

for r in results:
    cid = r["conjunction_id"]
    pc0 = r["Pc0"]

    for solver_name, out in r["solver_outcomes"].items():

        if not out.get("available"):
            rows_summary.append({
                "conjunction_id": cid,
                "solver": solver_name,
                "available": False
            })
```

```python
        else:
            rows_summary.append({
                "conjunction_id": cid,
                "solver": solver_name,
                "available": True,
                "Pc0": pc0,
                "Pc_after": out["pc_after"],
                "dv_total": out["dv_total"],
                "energy": out["energy"],
                "runtime_s": out["runtime_s"],
                "feasible": out["feasible"]
            })

summary_df = pd.DataFrame(rows_summary)

summary_df
```

Figure 6.1:Solver Evaluation Code

Step 7: Solver Performance Analysis and Metrics

Solver performance metrics were computed across all conjunction events. These included feasibility rate, mean Δv, runtime performance, and collision probability reduction.

A weighted ranking score was computed to identify the best solvers. The top-performing solvers were selected for further evaluation.

```python
metrics = []

for solver_name in summary_df["solver"].unique():

    sub = summary_df[
        (summary_df["solver"] == solver_name) &
        (summary_df["available"] == True)
    ]

    # If solver not available
    if len(sub) == 0:
        metrics.append({
            "solver": solver_name,
            "tested": 0,
            "feasible_rate": None,
            "mean_dv": None,
            "mean_runtime_s": None,
            "mean_pc_reduction": None,
            "mean_pc_rel_reduction": None
        })
        continue

    # Average DV
    mean_dv = sub["dv_total"].mean()

    # Runtime
    mean_runtime = sub["runtime_s"].mean()

    # Absolute Pc Reduction
    abs_red = (sub["Pc0"] - sub["Pc_after"]).mean()
```

```python
    # Relative Pc Reduction
    rel_red = (
        (sub["Pc0"] - sub["Pc_after"]) /
        sub["Pc0"].replace(0, np.nan)
    ).replace([np.inf, -np.inf], np.nan).mean()

    # Feasibility Rate
    feasible_rate = sub["feasible"].mean()

    metrics.append({
        "solver": solver_name,
        "tested": len(sub),
        "feasible_rate": float(feasible_rate),
        "mean_dv": float(mean_dv),
        "mean_runtime_s": float(mean_runtime),
        "mean_pc_reduction": float(abs_red),
        "mean_pc_rel_reduction": float(
            rel_red if not np.isnan(rel_red) else 0.0
        )
    })

# Convert to dataframe
metrics_df = pd.DataFrame(metrics)


# ==============================
# Add Ranking Score
# ==============================

metrics_df["score"] = (
    metrics_df["mean_pc_rel_reduction"].fillna(0) * 0.5 +
    metrics_df["feasible_rate"].fillna(0) * 0.3 -
    metrics_df["mean_runtime_s"].fillna(0) * 0.2
)
```

```python
# ================================
# Sort Best Solvers
# ================================

metrics_df = metrics_df.sort_values(
    "score",
    ascending=False
)


# ================================
# Select Top Solvers
# ================================

TOP_K = 7   # number of best solvers to keep

top_solvers = metrics_df.head(TOP_K)


# ================================
# Display Results
# ================================

print("===== Solver Ranking =====")
display(metrics_df)

print("\n===== Top Solvers =====")
display(top_solvers)
```

```python
# ==============================
# Save Results (Optional)
# ==============================

metrics_df.to_csv("solver_metrics.csv", index=False)
top_solvers.to_csv("top_solvers.csv", index=False)

metrics_df
```

Figure 7.1: Solver Performance Metrics & Solver Ranking

Step 8: Final Solver Selection

Top-performing solvers were selected based on ranking metrics. These solvers were evaluated again on high-risk conjunctions to determine final performance.

Final results were aggregated and exported for further analysis and the best solver was selected based on collision probability reduction and feasibility.

This final stage determined which conjunction events were manoeuvrable. Top solvers generated candidate manoeuvres, and feasible solutions were selected. The final dataset included conjunction ID, collision probability reduction, best solver, and required Δv. These results were exported for further simulation and decision support.

```python
# Select Top Solvers from Step 7
TOP_K = 5   # final model solvers

final_solver_list = top_solvers["solver"].head(TOP_K).tolist()

print("Final Selected Solvers:")
print(final_solver_list)


# ==============================
# Build Final Solver Mapping
# ==============================

FINAL_SOLVERS = {
    name: SOLVERS[name]
    for name in final_solver_list
    if name in SOLVERS
}

# ==============================
# Final Model Evaluation
# ==============================

final_results = []

for idx, row in cdm_sorted.iterrows():

    conj_id = row.get("conjunction_id", f"row{idx}")
    pc0 = float(row.get("Pc", 0.0))

    # Build BQM
    bqm = build_bqm_for_conjunction(pc0)
```

```python
    for solver_name, solver_fn in FINAL_SOLVERS.items():

        try:
            sol, runtime = solver_fn(bqm)

            if sol is None:
                continue

            sample = sol.sample
            sample_dict = dict(sample)

            # Compute DV
            dv_total = sum(
                var_to_dv[v] * int(sample_dict.get(v, 0))
                for v in var_names
            )

            # Compute Pc after
            pc_after = collision_probability_after(pc0, dv_total)

            feasible = dv_total <= MAX_DV_ALLOW

            energy = (
                float(sol.energy)
                if hasattr(sol, "energy")
                else bqm.energy(sample_dict)
            )

            final_results.append({
                "conjunction_id": conj_id,
                "solver": solver_name,
                "Pc0": pc0,
                "Pc_after": pc_after,
                "dv_total": dv_total,
                "runtime_s": runtime,
```

```python
                "energy": energy,
                "feasible": feasible
            })

        except Exception as e:
            continue


# ==============================
# Final Results DataFrame
# ==============================

final_df = pd.DataFrame(final_results)

print("Final Model Results:")
display(final_df)


# ==============================
# Final Performance Summary
# ==============================

final_summary = final_df.groupby("solver").agg({
    "Pc0": "mean",
    "Pc_after": "mean",
    "dv_total": "mean",
    "runtime_s": "mean",
    "energy": "mean",
    "feasible": "mean"
}).reset_index()

final_summary["Pc_reduction"] = (
    final_summary["Pc0"] - final_summary["Pc_after"]
)
```

```python
final_summary = final_summary.sort_values(
    "Pc_reduction",
    ascending=False
)

print("Final Model Summary:")
display(final_summary)


# ==============================
# Save Final Outputs
# ==============================

final_df.to_csv("fursan_final_qubo_results.csv", index=False)
final_summary.to_csv("fursan_final_qubo_summary.csv", index=False)


# ==============================
# Best Solver Selection
# ==============================

best_solver = final_summary.iloc[0]["solver"]

print("Best Final Solver:", best_solver)

best_solver
```

Figure 8.1: Final Solver Selections and Evaluations

QUBO Solver Evaluation

The QUBO solver evaluation assessed the performance of classical and quantum-inspired optimisation methods used for satellite manoeuvre planning in the FURSAN framework. The optimisation problem was formulated as a Quadratic Unconstrained Binary Optimisation (QUBO) model to determine the optimal collision-avoidance manoeuvre while minimising collision risk, fuel consumption, and operational constraints

$$\min_{x \in \{0,1\}^n} x^T Q x$$

Where x represents binary manoeuvre selection variables and Q encodes risk, fuel cost, and constraint penalties.

Solvers were compared using:

- Solution quality (objective energy minimisation)
- Runtime performance
- Constraint satisfaction
- Fuel efficiency (Δv usage)
- Collision risk reduction